

Laboratorio de Ataque Web — Paso a Paso

Este laboratorio tiene como objetivo proporcionar una experiencia práctica sobre cómo se manifiestan y se explotan vulnerabilidades web frecuentes en aplicaciones reales. A través de un entorno frontend/backend controlado, se reproducen 4 vulnerabilidades clave descritas por OWASP. Cada una es analizada y atacada con herramientas mínimas, como navegador, HTML, curl y scripts básicos.

Entorno del laboratorio

Se desarrollaron dos sistemas acoplados:

-  Un **frontend en Vue.js** simulando un panel administrativo (<http://localhost:5173>).
-  Un **backend en FastAPI** con conexión a SQLite (<http://localhost:8000>).

Ambos servicios trabajan de forma local y simulan fallas reales. Las vulnerabilidades se agrupan por casos:

Caso	Vulnerabilidad	Tecnología afectada
1	XSS reflejado	Vue + FastAPI
2	Exportación sin control de acceso	FastAPI
3	Dependencia vulnerable	Vue (jQuery)
4	CSRF	FastAPI (sin token)

Caso 1 – XSS reflejado en el campo de búsqueda

Componente afectado:

UserSearch.vue

Vector de ataque:

El input del usuario se inyecta sin escape usando `v-html`, permitiendo ejecutar scripts maliciosos directamente en el navegador.

```
<script>alert('XSS')</script>
```

Reproducción:

1. Abre el panel administrativo.
2. En el campo de búsqueda, ingresa el payload anterior.
3. Al hacer clic en "Buscar", el navegador ejecuta el código, confirmando la vulnerabilidad.

Archivo de ataque:

ataque/1_xss_reflejado.html

Este archivo simula la inyección desde una página externa, útil para demostrar el reflejo del XSS.

3 Caso 2 – Exportación sin autenticación

Endpoint vulnerable:

GET /admin/users/export

Observación:

El backend devuelve un archivo CSV con información de todos los usuarios sin requerir ningún tipo de autenticación o validación de rol.

Reproducción:

```
curl -X GET http://localhost:8000/admin/users/export
```

El archivo se devuelve correctamente incluso si no se está logueado, lo cual representa una fuga de datos grave.

 Archivo de ataque:

```
ataque/2_export_sin_auth.sh
```

Caso 3 – Uso de dependencia vulnerable

 Tecnología usada:

jQuery 1.12.4 fue instalado a propósito en el frontend (`npm install jquery@1.12.4`).

 Observación:

Esta versión de jQuery presenta múltiples vulnerabilidades conocidas y está listada como crítica en [Snyk](#).

 Reproducción:

```
curl -s http://localhost:5173 | grep jquery
```

También se puede usar Snyk o ZAP para escanear el frontend y detectar esta dependencia insegura.

 Archivo de ataque:

```
ataque/3_jquery_vulnerable_detect.sh
```

Caso 4 – CSRF en eliminación de usuarios

 Endpoint vulnerable:

```
POST /delete-user
```

 Vector de ataque:

La acción de eliminar usuario no requiere autenticación ni protección contra solicitudes forjadas. Un atacante puede crear una página externa que auto-envía el formulario sin consentimiento del usuario.

Reproducción:

1. Abre el archivo:
`ataque/4_csrf_delete.html`
2. Al cargar, el formulario se envía automáticamente.
3. Si el usuario 1 se elimina, la vulnerabilidad está confirmada.

Archivo de ataque:

`ataque/4_csrf_delete.html`

6 Registro de tráfico y observación en tiempo real

Durante los ataques, puedes observar el tráfico en consola mediante los logs de Uvicorn:

```
uvicorn main:app --reload --port 8000
```

Los logs como:

```
GET /search?q=<script>alert(1)</script> HTTP/1.1" 200
```

```
POST /delete-user HTTP/1.1" 200
```

...confirman que los ataques están siendo procesados por el backend sin filtros.

7 Análisis general

Este laboratorio demuestra cómo fallas pequeñas como:

- Falta de escape en frontend
- Ausencia de autenticación en backend
- Omisión de tokens de verificación
- Uso de dependencias obsoletas

...pueden combinarse para comprometer completamente un sistema.

Estas vulnerabilidades son altamente explotables y frecuentemente presentes en auditorías reales. Simularlas en un entorno controlado permite entender su impacto y anticipar medidas de mitigación que veremos en la siguiente etapa del laboratorio.

